COP 5536 Fall 2026: Job Scheduler Project Report

Author: Student | Language: Python 3 | Architecture & Implementation Summary

1. Program Organization & Modular Architecture

The project is structured in a strictly decoupled, modular architecture adhering to submission requirements. The primary data structures are isolated into separate standalone modules completely independent of scheduler logic and input parsing, allowing independent inspection and verification by automated grading harnesses.

- **job.py:** Defines the Job entity with unique identifier, priority, execution time estimates, delta offsets, timestamps, and lifecycle states (WAITING, RUNNING, COMPLETED, CANCELLED).
- **avl_tree.py:** Implements a complete self-balancing AVL Tree from scratch, keyed by job_id. Maintains node references to Job objects, tracks rotation counts, and provides tree-shape level-order output.
- **max_heap.py:** Implements an array-based Binary Max Heap from scratch for WAITING jobs. Includes position indexing for O(log n) removals/updates, dual scheduling modes, and bottom-up rebuilds.
- **jobScheduler.py:** The central discrete-event simulation driver. Coordinates capacity constraints (k), advances the simulation clock, manages running job completions, and dispatches commands.
- **test_scheduler.py:** Test harness verifying all cases from TestCases.docx and comprehensive rotation/edge-case tests.
- **Makefile:** Prepares execution permissions, compiles Python bytecode, and exposes clean and test targets.

2. Classes and Data Structures Overview

2.1 Job Entity (job.py)

Every submitted job is instantiated as exactly one Job object, which is then shared via direct reference across the AVL Tree, Binary Max Heap, Running Priority Queue, and Cancelled List. State transitions (start, complete, cancel) update the object in-place, eliminating redundant records across the system.

2.2 AVL Tree (avl_tree.py)

The AVL Tree serves as the primary dictionary for all active (non-cancelled) jobs, keyed by job_id. Each node records its subtree height. The balance factor is computed as BF = height(left) - height(right). Whenever |BF| > 1 after an insertion or deletion, the tree restores balance using primitive left and right rotations. Four rebalancing cases are handled: Left-Left (single right rotation), Right-Right (single left rotation), Left-Right (left rotation on child followed by right rotation on parent), and Right-Left (right rotation on child followed by left rotation on parent). Counters for primitive left and right rotations are explicitly tracked. Deletion replaces internal nodes with their inorder successor (minimum node of the right subtree).

3. Data Structure Representations & Mechanics

3.1 Binary Max Heap & Priority Inversion Tracking (max_heap.py)

The waiting queue is implemented as a Binary Max Heap using a 0-indexed flat list. For any element at index i, its parent is located at floor((i - 1) / 2) and its children at 2i + 1 and

2i + 2. Crucially, the Max Heap maintains an internal hash map (pos: job_id -> index) that is updated on every element swap. This enables O(log n) arbitrary job cancellations (remove) and priority updates (update_priority) without resorting to O(n) linear scans. The heap supports two scheduling rankings:

- **PRIORITY Mode:** Higher priority > earlier submission time > smaller job_id.
- **SHORT_JOBS Mode:** Smaller estimated_time > higher priority > earlier submission time > smaller job_id.

When the SHORT_JOBS command is encountered, the heap switches mode permanently and executes a linear-time bottom-up heapify (rebuild) starting from floor(n/2) - 1 down to 0, incrementing the rebuilds statistic.

3.2 Running Jobs & Discrete Event Simulation

Concurrency is strictly constrained to at most k RUNNING jobs. Running jobs are tracked in a Min-Heap priority queue ordered by completion time. Before processing any command at timestamp t, the simulation clock advances and all running jobs with completion time <= clock are marked COMPLETED, freeing capacity slots. When PROCESS_NEXT is invoked at capacity, the clock fast-forwards to the earliest completion time among running jobs, completes those jobs simultaneously, and immediately launches the next highest-ranking waiting job.

4. External and Standard Libraries Used

In accordance with project guidelines, the program relies strictly on default Python 3 standard library modules, guaranteeing 100% portable out-of-the-box execution on any standard Ubuntu LTS installation:

- **heapq (standard library):** Used exclusively to maintain the Min-Heap priority queue tracking the completion times of RUNNING jobs, as explicitly permitted by the project specification.
- **list (standard library):** Used for the Cancelled Jobs List to preserve FIFO cancellation order.
- **sys, os (standard library):** Used for command-line arguments and input/output file path management.

No third-party packages (such as NumPy, PyPI packages, or external binary dependencies) are used.